Advanced Programming in Python

Unit-1: Python Basics

Dr. Nemi Chandra Rathore
Department of Computer Science
Central University of South Bihar, Patna

July 23, 2024



Overview

1. Python Basics
   - About Python
   - Python Tokens
   - Python Control Structures
2. Exception Handling



Python: Origins and Early Development [5]

- Python was created by Guido van Rossum, and officially released as Python 0.9.0 in 1991.
- It included many **core features** of the language today, such as exception handling, functions, and the core data types (e.g., list, dict, str).
- Inspired by ABC, a teaching language developed at CWI (Centrum Wiskunde & Informatica) in the Netherlands.
- It, named after the British comedy group Monty Python, is an interpreted, interactive, object-oriented programming language.
- Python 1.0 was released in 1994 which included more features, such as lambda, map, filter, and reduce, inspired by functional programming languages like Lisp.



Python: Origins and Early Development [5]

- Python 2.0 was released in 2000 which reached its end of life on January 1, 2020.
- Python 3.0 was released in 2008 was designed to rectify fundamental design flaws in the language. This version was not backward compatible with Python 2.x.
- For example in Python 2, print was a statement not a function.
- For dividing two integers it used floor division by default, which could lead to unexpected results.



Python Basics: Introduction

- Python is maintained by the Python Software Foundation (PSF), a non-profit membership organization (established in 2000) and a community devoted to developing, improving, expanding, and popularizing the Python language and its environment [3].
- Python is a powerful and versatile programming language that can be used for various purposes.
- The availability of a sizable standard library and several third-party packages via the Python Package Index accelerated Python's acceptance.



Python: Origins and Early Development [5]

- Python continues to grow in popularity, particularly in areas like data science, machine learning, web development, and automation.
- Python's extensive standard library and the availability of powerful third-party libraries like NumPy, pandas, TensorFlow, Django, and Flask have made it a go-to language for various applications.
- Python's development is driven by its community, with contributions from numerous developers worldwide.
- The PSF oversees the language's development, ensuring it evolves to meet the needs of its users.



Python: Who uses Python? [5]

- Google uses Python for many internal tools and some of their services.
- Facebook utilizes Python for various infrastructure management tasks.
- Instagram built its backend on Python and Django.
- Spotify uses Python for data analysis and backend services
- Python is used in some of the largest Internet sites on earth - like Youtube, Dropbox, and Reddit.
- The popular Python web framework Django powers both Instagram and Pinterest.
- LucasFilms award winning visual effects company (USA based), Industrial Light & Magic, uses Python to make their magic come to life.



Python: Who uses Python? [5]

- J.P. Morgan uses Python for data analysis, trading algorithms, and risk management.
- Goldman Sachs employs Python for modeling, trading, and risk management systems.
- NASA uses Python for scientific computing and data analysis.
- CERN employs Python for data analysis in particle physics research.
- Johnson & Johnson uses Python for data analysis and predictive modeling.
- Pfizer employs Python for clinical data analysis and research.



Python: Who uses Python? [5]

- Netflix uses Python for content recommendation algorithms and backend services.
- Disney utilizes Python in animation and data analysis.
- OpenAI develops AI models and tools using Python.
- IBM uses Python for machine learning and AI projects.
- Uber employs Python for data analysis and machine learning models.
- CIA uses Python for data analysis and various software projects.
- NSA employs Python for cybersecurity and data analysis tasks.



Python: Who uses Python? [5]

Key Python Use Cases:

- Web Development: Frameworks like Django and Flask.
- Data Science and Analytics: Libraries like pandas, NumPy, and SciPy.
- Machine Learning: Libraries like TensorFlow, Keras, and Scikit-learn.
- Automation and Scripting: Widely used for writing scripts to automate repetitive tasks.
- Scientific Computing: Libraries like SciPy and SymPy.
- Game Development: Libraries like Pygame.
- Network Programming: Libraries like Twisted and asyncio.



Python Basic: Python Features

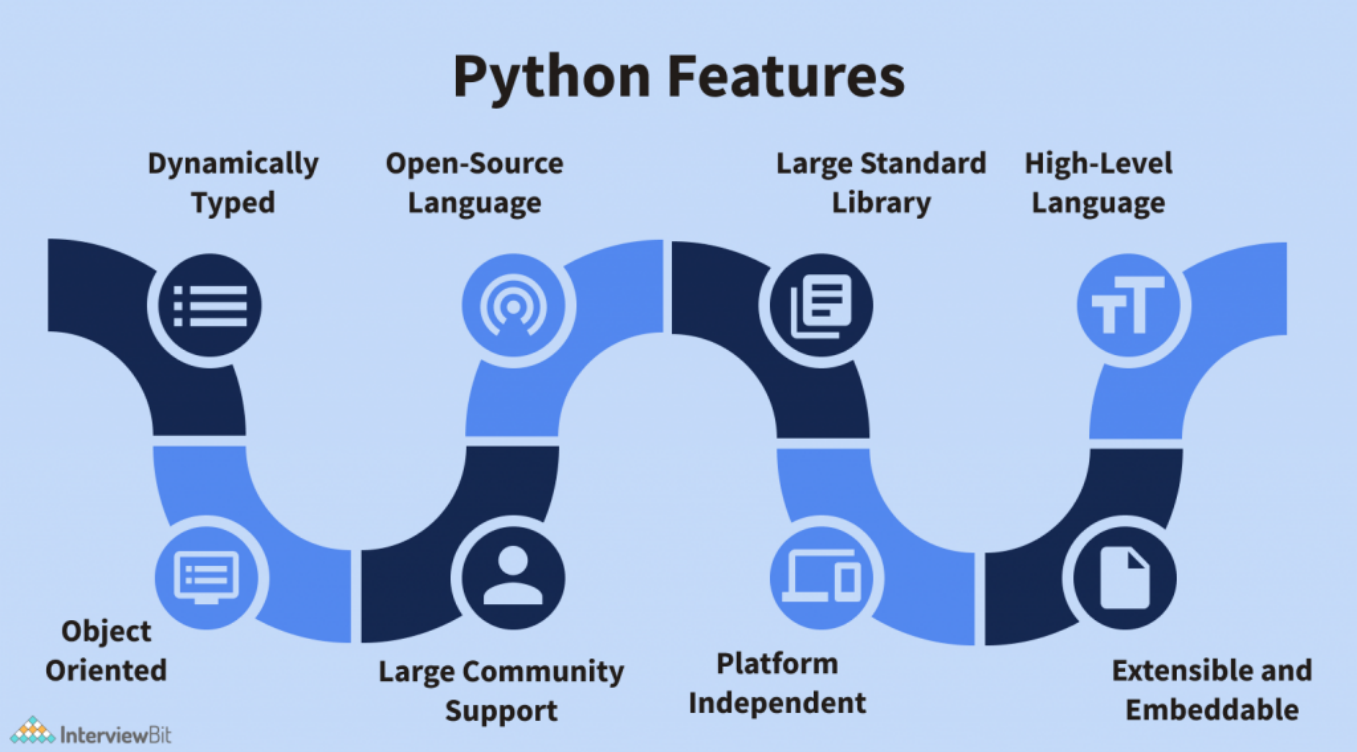


Figure 1: Features of Python [Img Src: https://www.interviewbit.com/blog/features-of-python]

Python Basics: Python Features [5]

Python is:

1. Easy To Learn and Readable language
2. Interpreted
3. Strongly typed: It enforces data types so you can't concatenate a string and an integer, for example.
4. Dynamically, implicitly typed: So, you don't have to explicitly declare variable data types. Data types are enforced at run-time.
5. Case sensitive: For example, *token* and *TOKEN* are two different variables.
6. High-level language
7. Object-oriented: Everything is an object.



Python Basic: Python Features [5]

8. Open Source And Free
9. Large Standard Library
10. Large Community Support
11. Platform Independent
12. Expressive: Python needs to use only a few lines of code to perform complex tasks
13. Extensible and Embeddable:
    - Python is an Embeddable language ie we can write some Python code into C or C++ language and also we can compile that code in C/C++ language.
    - Python is also extensible that is we can extend our Python code in various other languages like C++, etc. too.
14. Graphical User Interface (GUI) Support: Tkinter, PyQt, wxPython, or PySide.



Python Basics: Python limitations

- Python may not be a good choice for some application areas.
- Some examples are real-time systems, resource-intensive software, mobile app development, high-frequency trading, system-level programming, memory-intensive software, mobile or console gaming, and certain cryptographic algorithms[4].



Python Basics: Python limitations

- Python is not commonly used for mobile app development.
- Python is not well-suited for low-level programming tasks such as operating system kernels, device drivers, or embedded systems.
- Python can consume more memory than languages like C, C++ due to its high-level data structures and dynamic typing.
- Python's concurrency model is limited.



Python Basics: Variables [11]

- Variables are used to store data.
- Python determines the type of a variable based on the value assigned.
- Python has no command for declaring a variable.
- A variable is created the moment you first *assign a value* to it.
- If you want to specify the data type of a variable, this can be done with casting.

```
1 x = 5          # An integer variable
2 y = 3.14       # A floating-point variable
3 name = "Alice"  # A string variable
4 is_active = True  # A boolean variable
5 x=str(3) # (x will be 3)
6 y=float(3) # (x will be 3.0)
```



Python Basics: Types of Variables

- Integer (`int`): Whole numbers.
- Floating-point (`float`): Numbers with a fractional part.
- String (`str`): Sequence of characters.
- Boolean (`bool`): `True` or `False`.
- List (`list`): Ordered, mutable collection.
- Tuple (`tuple`): Ordered, immutable collection.
- Dictionary (`dict`): Key-value pairs.
- Set (`set`): Unordered collection of unique items.



Python Basics: Example of Variable Types

```
1 x = 10              # int
2 pi = 3.14159        # float
3 name = "John"       # str
4 flag = True         # bool
5 colors = ["red", "green", "blue"]  # list
6 coordinates = (10.0, 20.0)  # tuple
7 person = {"name": "Alice", "age": 30}  # dict
8 unique_numbers = {1, 2, 3, 4, 5}  # set
```



Python Basics: Variables [11]

Rules for naming Python variables:

- A variable name must start with a letter or the underscore character
- Subsequent characters can be letters, underscores, or digits.
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- A variable name cannot be any of the Python keywords.



Python Basics: Variable Naming Conventions

Example:

```
1  # Valid variable names
2  age = 25
3  _age = 25
4  age_2 = 25
5  # Invalid variable names
6  2age = 25     # Cannot start with a digit
7  age-2 = 25    # Cannot contain a hyphen
```



Python Basics: Variable Scope

- Local Scope: Variables defined within a function.
- Global Scope: Variables defined outside any function.

```
1   # Global variable
2   x = 10
3   def example_function():
4     # Local variable
5     y = 5
6     print(y)  # Accessing local variable
7   print(x)  # Accessing global variable
8   example_function()
9   print(x)  # Accessing global variable
10  print(y)  # Error: y is not defined outside the
    function
11
```



Python Basics: Multiple Variable Assignment

- Multiple variables can be assigned in a single statement.

```
1    a, b, c = 1, 2, 3
2    print(a)  # 1
3    print(b)  # 2
4    print(c)  # 3
5    # Swapping variables
6    a, b = b, a
7    print(a)  # 2
8    print(b)  # 1
9
```



Python Basics: Deleting Variables

- Variables can be deleted using the del keyword.

```
1    x = 10
2    print(x)  # 10
3    del x
4    print(x)  # Error: x is not defined
5
```



Variables: Global Keyword

- Modify a global variable inside a function using global.

```
1
2   x = 10
3   def modify_global():
4     global x
5     x = 20
6   modify_global()
7   print(x)  # 20
```



Variables: Nonlocal Keyword

- You can work with variables inside nested functions using nonlocal.

```
1     def outer_function():
2       x = 10
3       def inner_function():
4         nonlocal x
5         x = 20
6       inner_function()
7       print(x)  # 20
8     outer_function()
9
```



Constants

- Variables whose values should not change.
- Written in uppercase letters with underscores.

```
1     PI = 3.14159
2     GRAVITY = 9.8
3
```



Python Basics: print() function [11]

- The print() function in Python is used to output text/variables to the console.
- It is one of the most commonly used functions in Python.
- In the print() function, you output multiple variables, separated by a comma;
- Example: $print(x, y, z)$



Python Basics: print() function

- The syntax is:
  'print(object(s), sep=separator, end=end, file=file, flush=flush)'
- Example:

```
1       print("Hello, World!")
2
```

- This will output: `Hello, World!`



Python Basics: print() function

- You can print multiple objects by separating them with commas.
- Example:

```
1       print("Hello,", "World!")
2
```

- This will output: `Hello, World!`



Python Basics: print() function

- The sep parameter specifies the separator between objects.
- Example:

```
1       print("Hello", "World", sep="-")
2
```

- This will output: `Hello-World`



Python Basics: print() function

- The end parameter specifies what to print at the end.
- Default is newline.
- Example:

```
1       print("Hello, World!", end=" END")
2
```

- This will output: `Hello, World!  END`



Python Basics: print() function

- The file parameter specifies the file to write to.
- By default, it is ‘sys.stdout‘.
- Example:

```
1       with open("output.txt", "w") as f:
2       print("Hello, World!", file=f)
3
```

- This writes ‘Hello, World¡ to ‘output.txt‘.



Python Basics: print() function

- The flush parameter specifies whether to forcibly flush the stream.
- Example:

```
1       print("Hello, World!", flush=True)
2
```

- This ensures the output is written immediately.



Python Basics: print() function

- The print() function is versatile and useful for debugging and displaying information.
- Understanding its parameters can help in customizing the output.



Python Basics: Variables [11]

- You can get the data type of a variable with the type() function. Example: print(type(x))
- String variables can be declared either by using single or double quotes. For example: $name = "Ram"$ and $name = 'Ram'$ both are equivalent.
- Variable names are case-sensitive.
- Python allows you to assign values to multiple variables in one line: Example: $x, y, z = "Orange", "Banana", "Cherry"$
- If you have a collection of values in a list, tuple etc., Python allows you to extract the values into variables. This is called unpacking
- $fruits = ["apple", "banana", "cherry"]$
- $x, y, z = fruits$



Python Operators

Python divides the operators in the following groups:

1. Arithmetic operators
2. Comparison operators
3. Logical operators
4. Identity operators
5. Membership operators
6. Bitwise operators
7. Assignment operators



Python Operators:Arithmetic operators

Allows us to perform arithmetic operations:

Operator	Example	Syntax
$+$	Addition	$x + y$
$-$	Subtraction	$x - y$
$*$	Multiplication	$x * y$
$/$	Division	x/y
%	Modulus	$x\%y$
$**$	Exponentiation	$x * *y$
$//$	Floor division[1]	$x//y$

Table 1: Arithmetic operators

[1] Integer division



Python Operators: Relational operators

Allows us to perform relational operations over variables and constants:

Operator	Example	Syntax
==	Equal	$x == y$
! =	Not equal	$x! = y$
>	Greater than	$x > y$
<	Less than	$x < y$
>=	Greater than or equal to	$x >= y$
<=	Less than or equal to	$x <= y$

Table 2: Relational operators



Python Operators: Logical operators

Allows us to perform logical operations over logical variables and constants:

Operator	Description	Example
and	Returns True if both statements are true	$x < 5$ *and* $x < 10$
or	Returns True if one of the statements is true	$x < 5$ *or* $x < 4$
not	Reverse the result, returns False if the result is true	*not* $(x < 5$ *and* $x < 10)$

Table 3: Logical operators



Python Operators: Identity Operators

Allows us to check whether two objects are same or not:

Operator	Description	Example
is	Returns true if both variables are the same object	x is y
is not	Returns true if both variables are not the same object	x is not y

Table 4: Identity operators



Python Operators: Membership Operators

Allows us to check membership of an object in a list/collection/sets of objects:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object x not in y	x not in y

Table 5: Membership operators



Python Operators: Bitwise operators

Allows us to perform bitwise operations over binary numbers:

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the left-most bit in from the left, and let the rightmost bits fall off

Table 6: Bitwise operators



Python Operators:Assignment operators

Allows us to perform assignments of values to different variables :

Operator	Syntax	Equivalent Form
$=$	$x = 5$	$x = 5$
$+=$	$x+ = 3$	$x = x + 3$
$-=$	$x- = 3$	$x = x - 3$
$*=$	$x* = 3$	$x = x * 3$
$\%=$	$x\% = 3$	$x = x\%3$
$/=$	$x/ = 3$	$x = x/3$
$**=$	$x** = 3$	$x = x**3$
$\&=$	$x\& = 3$	$x = x\&3$
$\|=$	$x\| = 3$	$x = x\|3$
$\hat{}=$	$x\hat{} = 3$	$x = x\hat{}3$
$>>=$	$x >>= 3$	$x = x >> 3$
$<<=$	$x <<= 3$	$x = x << 3$

Table 7: Assignment operators



Python Basics: Control Structures

- An "if statement" is written by using the if keyword.
- Python relies on indentation (whitespace at the beginning of a line) to define scope in the code.

```
if b > a:
        print("b is greater than a")
```

- The elif keyword is Python's way of saying "if the previous conditions were not true, then try this condition". Example:

```
if b > a:
        print("b is greater than a")
elif a == b:
        print("a and b are equal")
```



- The else keyword catches anything that isn't caught by the preceding conditions.

```
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
else:
    print("a is greater than b")
```

- If you have only one statement to execute, you can put it on the same line as the if statement. Example:

```
if a > b: print("a is greater than b")
```



Python Basics: Control Structures

- If you have only one statement to execute, one for if, and one for else, you can put it all on the same line: Example:

```
print("A") if a > b else print("B")
```

- You can have if statements inside if statements, this is called nested if statements.

```
if x > 10:
    print("Above ten,")
    if x > 20:
        print("and also above 20!")
    else:
        print("but not above 20.")
```



- If statements cannot be empty, but if you for some reason have an if statement with no content, put in the pass statement to avoid getting an error. Example:

```
if b > a:
    pass
```



Python Basics: Control Structures

- Python has two primitive loop commands:
  1. while loops
  2. for loops
- With the while loop you can execute a set of statements as long as a condition is true. Example:

```
i = 1
while i < 6:
    print(i)
    i += 1
```

- With the break statement we can stop the loop even if the while condition is true.
- With the continue statement we can stop the current iteration, and continue with the next.



Python Basics: Control Structures

- With the else statement with the loop you can run a block of code once when the condition no longer is true. Example:

```
i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```



Python Basics: Control Structures

- A for loop is used for *iterating over* a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- In this loop you can execute a set of statements, once for *each item* in a list, tuple, set etc. Example:

```
fruits = ["apple", "banana", "mango"]
for x in fruits:
    print(x)
```

- Even strings are iterable objects, they contain a sequence of characters. Example:

```
for x in "banana":
    print(x)
```



Python Basics: Control Structures

- To loop through a set of code a specified number of times, we can use the range() function
- This function returns a sequence of numbers, starting from 0 by default, increments by 1 (by default), and ends at a specified number (Excluding the number). Example:

```
for x in range(6):
    print(x)
```

- Note that range(6) is not the values of 0 to 6, but the values 0 to 5.



Python Basics: Control Structures

- The range() function defaults to increment the sequence by 1, however, it is possible to specify the increment value by adding a third parameter, for example: range(2, 30, 3)
- The else keyword in a for loop specifies a block of code to be executed when the loop is finished:

```
for x in range(6):
    print(x)
else:
    print("Finally finished!")
```

- Note: The else block will NOT be executed if the loop is stopped by a break statement.
- For some reason have a for loop with no content, put in the pass statement to avoid getting an error.



Python Basics: Data Types

Some built-in Python data types are [9]:

1. Numeric data types: int, float, complex
2. String data types:
3. Sequence types: list, tuple, range
4. Binary types: bytes, bytearray, memoryview
5. Mapping data type: dict
6. Boolean type: bool
7. Set data types: set, frozenset



Python Basics: Data Types [9]

1. Numeric data types:
   1. int - holds signed integers of non-limited length.
   2. long- holds long integers (exists in Python 2.x, deprecated in Python 3.x).
   3. float- holds floating precision numbers and it's accurate up to 15 decimal places. Float can also be scientific numbers with an "e" to indicate the power of 10.
   4. complex- holds complex numbers. They are written with a "j" as the imaginary part Example: $x = 3 + 5j$ or $y = 5j$ or $z = -5j$
   5. You can convert from one type to another with the int(), float(), and complex() methods:



Python Data Types: Strings [11]

2. Strings
   - Strings in Python are arrays of bytes representing Unicode characters[2].
   - Strings are represented by either single or double quotes.
   - Python supports Unicode characters.
   - Note: 'hello' is the same as "hello".
   - You can assign a multiline string to a variable by using three quotes:

```
a = """Like many other popular programming languages, strings in Python are arrays of bytes representing Unicode characters. However, Python does not have a character data type, a single character is simply a string with a length of 1. Square brackets can be used to access elements of the string."""
print(a)
```

[2] Note: Python does not have a character data type, a single character is simply a string with a length of 1



Python Data Types: Strings [11]

- To check if a certain phrase or character is present in a string, we can use the keyword in. For Example, Check if "free" is present in the following text:

```
txt = "The best things in life are free!"
print("free" in txt)
```

- To concatenate, or combine, two strings you can use the + operator.

```
a = "CUSB"
b = " Gaya"
c = a + b
print(c)
```



Python Data Types: Strings [11]

- Strings Functions

Table 8: Common functions for working with strings

Function	Description
len(str)	Given the length of string *s*
str.capitalize()	Converts the first character of the string *str* in the string to upper case
str.upper()	Converts *str* string into upper case
str.title()	Converts the first character of each word of *str* to upper case
str.strip()	Remove any whitespace[3] from the string.
str.replace(*x*,*y*)	Replaces string *x* with string *y* in *str*:
str.split()(*y*)	Splits the string *str* into sub-strings if it finds instances of the separator *y*
str.count(*y*, *[start]*, *[end]*)	Count all the occurrences of *y* in *str*
str.casefold()/ str.lower()	Returns a string where all the characters are lowercase



[3]Whitespace is the space before and/or after the actual text

Python Data Types: Strings [11]

- Strings Functions

Table 10: Common functions for working with strings

Function	Description
str.center()	Center align the string, using a specified character (space is the default) as the fill character.
str.find(*value*, *start*, *end*)	Finds the first occurrence of the specified value. Returns -1 if *value* not found.
str.isalnum()	Returns True if all the characters are alphanumeric, meaning alphabet letters (a-z) and numbers (0-9).
str.isascii()	Returns True if all characters in the string are ASCII characters.
str.isdecimal()	Returns True if all characters in the string are decimals.
str.isdigit()	Returns True if all characters in the string are digits.



Python Data Types: Strings [11]

- Strings Functions

Table 12: Common functions for working with strings

Function	Description
str.islower()	Returns True if all characters in the string are lowercase.
str.isnumeric()	Returns True if all characters in the string are numeric.
str.isprintable()	Returns True if all characters in the string are printable[4].
str.isidentifier()	Returns True if the string is an identifier.
str.isspace()	Returns True if all characters in the string are whitespaces.
str.isupper()	Returns True if all characters in the string are upper case.
str.istitle()	Returns True if all words in *str* start with capital.



[4] Example of none printable character can be carriage return and line feed.

Python Data Types: Strings [11]

- Strings Functions

Table 14: Common functions for working with strings

Function	Description
str.swapcase()	Make the lower case letters upper case and the upper case letters lower case.
str.join(iterable[5])	takes all items in an iterable and joins them into *str*.
str.rstrip(characters)	Removes any trailing characters (characters at the end of a string), space is the default trailing character to remove.
str.lstrip(characters)	Removes any leading characters (characters at the end of a string), space is the default trailing character to remove.



[5] An iterable object where all the returned values are strings

3 Sequence Types

- List:
  - List is an ordered sequence of some data written using square brackets([,]) and commas(,).
  - Lists are used to store multiple items in a single variable.
  - It is the same as the array in C/C++ but the difference is that it can simultaneously hold *different types of data*.
  - Examples: $a = [1, 2, 3, 4, 5]$ and Name=["Ram", "Shyam", "Ravi", "Mathew", 4, 100].
  - List items are ordered, changeable, and allow duplicate values.
  - List items are indexed, the first item has index [0], the second item has index [1] and so on.
  - If you add *new items* to a list, the new items will be placed *at the end* of the list.
  - The list is changeable, meaning that we can change, add, and remove items in a list after it has been created



Python Sequence Data Types: Lists [11]

- To determine how many items a list *mylist* has, use the len(mylist) function.
- From Python's perspective, lists are defined as objects with the data type 'list'.

```
1 mylist = ["apple", "banana", "cherry", 1000,
      230.45]
2 print(type(mylist))
3 print(len(mylist))
```

- It is also possible to use the *list() constructor* when creating a new list.

```
1 mylist = list(("apple", "banana", "cherry"))
2 print(mylist)
```



Python Sequence Data Types: Lists [11]

- List items are indexed and you can access them by referring to the index number.
- Python allows negative indexing for lists. means start from the end.
- -1 refers to the last item, -2 refers to the second last item and so on.

```
1 mylist = ["apple", "banana", "cherry", 1000,
      230.45]
2 print(mylist[0])
3 print(mylist[1])
4 print(mylist[-1])
5 print(mylist[-2])
```



Python Sequence Data Types:: Lists [11]

- Range of Indexes: You can specify a range of indexes by specifying where to start and where to end the range.
- When specifying a range, the return value will be a new list with the specified items. Example:

```
1 thislist = ["apple", "banana", "cherry", "orange",
      "kiwi", "melon", "mango"]
2 print(thislist[2:5])
3 print(thislist[:4])
4 print(thislist[2:])
5 print(thislist[-4:-1])
```

- Note: The search will start at index 2 (included) and end at index 5 (not included).
- By leaving out the start value, the range will start at the first item.



Python Sequence Data Types: Lists [11]

- By leaving out the end value, the range will go on to the end of the list.
- Specify negative indexes if you want to start the search from the end of the list.

```
1 mylist = ["apple", "banana", "cherry", "orange", "
    kiwi", "melon", "mango"]
2 print(mylist[-4:-1])
3 if "apple" in mylist:
4   print("Yes, 'apple' is in the fruits list")
```

- In the above example data item at index -1 is not included but at index -4 is included.
- To determine if a specified item is present in a list use the in keyword.



Python Sequence Data Types: Lists [11]

- To change the value of a specific item, refer to the index number. Example

```
1 mylist = ["apple", "banana", "cherry"]
2 mylist[1] = "mango"
3 print(mylist)
```

- To change the value of items within a specific range, define a list with the new values and refer to the range of index numbers where you want to insert the new values. Example:

```
1 mylist = ["apple", "banana", "cherry", "orange", "
      kiwi", "mango"]
2 mylist[1:3] = ["blackcurrant", "watermelon"]
3 print(mylist)
```



Python Sequence Data Types: Lists [11]

- If you insert more items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly. Example:

```
1 mylist = ["apple", "banana", "cherry", "orange", "
    kiwi", ]
2 mylist[1:2] = ["blackcurrant", "watermelon"]
3 print(mylist)
```

- If you insert fewer items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly: Example:

```
1 mylist = ["apple", "banana", "cherry", "orange", "
    kiwi", "mango"]
2 mylist[1:3] = [ "watermelon"]
3 print(mylist)
```



Python Sequence Data Types: Lists [11]

- The insert() method inserts an item at the specified index. Example:

```
1 mylist = ["apple", "banana", "cherry", "orange", "
    kiwi"]
2 mylist.insert(2, "watermelon")
3 print(mylist)
```

- To add an item to the end of the list, use the append() method. Example:

```
1 mylist = ["apple", "banana", , "kiwi", "mango"]
2 mylist.append("orange")
3 print(mylist)
```



Python Sequence Data Types: Lists [11]

- To append elements from *another list* to the *current list*, use the extend() method Example:

```
1 mylist = ["apple", "banana", "cherry", "orange", ]
2 tropical = ["mango", "pineapple", "papaya"]
3 mylist.extend(tropical)
4 print(mylist)
```

- The extend() method does not have to append lists, you can add any iterable object (tuples, sets, dictionaries etc.)

```
1 mylist = ["apple", "banana", , "kiwi", "mango"]
2 mytuple = ("papaya", "orange")
3 thislist.extend(mytuple)
4 print(mylist)
```



Python Sequence Data Types: Lists [11]

- The remove() method removes the specified item. Example:

```
1 mylist = ["apple", "banana", "cherry", "orange", ]
2 mylist.remove("banana")
3 print(mylist)
4 }
```

- If there are more than one item with the specified value, this method removes the first occurrence.
- The pop() method removes the specified index.

```
1 mylist = ["apple", "banana", , "kiwi", "mango"]
2 mylist.pop(1)
3 print(mylist)
```

- If you do not specify the index, then it removes the last item.



Python Sequence Data Types: Lists [11]

- The del() keyword also removes the specified item. Example:

```
1 mylist = ["apple", "banana", "cherry", "orange" ]
2 del mylist[0]
3 print(mylist)
```

- The del keyword can also delete the list completely. Example:

```
1 mylist = ["apple", "banana", , "kiwi", "mango"]
2 del mylist
3 print(mylist)
```

- The clear() method empties the list. Example:

```
1 listname.clear()
```



Python Sequence Data Types: Looping through a list [11]

- You can loop through the list items by using a for loop. Example:

```
1 thislist = ["apple", "banana", "cherry", "orange"]
2 for x in thislist):
3   print(x)
```

```
1 thislist = ["apple", "banana", "cherry", "orange"]
2 for i in range(len(thislist)):
3   print(thislist[i])
```



Python Sequence Data Types:: Looping through a list [11]

- You can loop through the list items by using a while loop. Example

```
1 thislist = ["apple", "banana", "cherry", "orange"
      ]
2 i=0
3 while i < len(thislist):
4   print(thislist[i])
5   i = i + 1
```



Python Sequence Data Types: Looping Using List Comprehension [11]

- List Comprehension offers the shortest syntax for looping through lists. Example:

```
1  thislist = ["apple", "banana", , "kiwi", "mango"
     ]
2  [print(x) for x in thislist]
```

- It offers a shorter syntax when you want to create a new list based on the values of an existing list. For example, based on a list of fruits, you can create a new list, containing only the fruits with the letter "a" in the name as follows:

```
1  fruits = ["apple", "banana", "cherry", "kiwi", "
     mango"]
2  newlist = [x for x in fruits if "a" in x]
3  print(newlist)
```



Python Sequence Data Types:: Looping Using List Comprehension [11]

- Without list comprehension you will have to write a for statement with a conditional test inside

```
1  fruits = ["apple", "banana", "cherry", "kiwi", "
     mango"]
2  newlist = [ ]
3  for x in fruits:
4    if "a" in x:
5      newlist.append(x)
6  print(newlist)
```

- Syntax:
  newlist = [expression for *item* in iterable if condition == True]
- The condition is optional and can be omitted. Example:

```
1  fruits = ["apple", "banana", "cherry", "kiwi", "
     mango"]
2  newlist = [x for x in fruits]
3  print(newlist)
```



Python Data Types: List Sorting [11]

- Sorting a List: List objects have a sort() method that will sort the list alphanumerically, ascending, by default.
- To sort descending, use the keyword argument reverse = True:

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
fruits.sort()
fruits.sort(reverse = True)
```

- By default the sort() method is case-sensitive, resulting in all capital letters being sorted before lowercase letters.



Python Data Types: List Sorting [11]

- You can also customize your own function by using the keyword argument key = function.
- The function will return a number that will be used to sort the list.
- For example, Sort the list based on how close the number is to 50.

```
def myfunc(n):
      return abs(n - 50)
thislist = [100, 50, 65, 82, 23]
thislist.sort(key = myfunc)
print(thislist)
```



Python Data Types: List Sorting [11]

- By default the sort() method is case sensitive, resulting in all capital letters being sorted before lowercase letters which can give unexpected results sometimes. For example:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.sort()
print(thislist)
```

- Luckily we can use built-in functions as key functions when sorting a list. For example:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.sort(key = str.lower)
print(thislist)
```



Python Data Types: Reversing a List [11]

- Reversing a List: The reverse() method reverses the current sorting order of the elements. Example:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.reverse()
print(thislist)
```



Python Data Types: Copying a List [11]

- To make a copy of a list we can use copy() method. For example:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
mylist = thislist.copy()
print(mylist)
```

- Another way to make a copy is to use the built-in method list(). For example:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
mylist = list(thislist)
print(mylist)
```



Python Data Types: Joining Lists [11]

- Joining two Lists: There are several ways to join, or concatenate, two or more lists in Python.
- One of the easiest ways is by using the + operator. Example:

```
tlist1 = ["banana", "Orange", "Kiwi", "cherry"]
list2 = [1, 2, 3]
list3 = list1 + list2
print(list3)
:: Append list2 with list1
for x in list2:
      list1.append(x)
print(list1)
```

- Another way to join two lists is by appending all the items from list2 into list1, one by one



Python Data Types: Joining Lists [11]

- you can also use the extend() method for this purpose.

```
list1 = ["banana", "Orange", "Kiwi", "cherry"]
list2 = [1, 2, 3]
list1.extend(list2)
print(list1)
```

- You can use clear() method to remove all elements from the given list. Example

```
fruits = ['apple', 'banana', 'cherry', 'orange']
fruits.clear()
```



Python Data Types: Tuples [11]

- A tuple is a collection which is ordered[6] and unchangeable[7].
- Tuples allow duplicate values.
- Tuple items are indexed.

```
thistuple = ("apple", "banana", "cherry", "apple", "cherry")
print(thistuple)
print(len(thistuple))
```

- To determine how many items a tuple has, use the len() function.

[6] It means that the items have a defined order, and that order will not change.

[7] It means that we cannot change, add or remove items after the tuple has been created.



Python Data Types: Tuples [11]

- To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.
- Tuple items can be of any data type.
- From Python's perspective, tuples are defined as objects with the data type 'tuple'.
- It is also possible to use the tuple() constructor to make a tuple. Example:

```
thistuple = tuple(("apple", "banana", "cherry"))
print(thistuple)
print(type(thistuple))
```



Python Data Types: Tuples [11]

- You can access tuple items by referring to the index number (starting with 0) (similar to a list), inside square brackets.
- Python also supports negative indexes for tuples.

```
thistuple = tuple(("apple", "banana", "cherry"))
print(thistuple[1])
print(type(thistuple))
```

- Tuples are unchangeable, or immutable.
- However, you can convert the tuple into a list, change the list, and convert the list back into a tuple.



Python Data Types: Tuples [11]

- Add tuple to a tuple: You are allowed to add tuples to tuples, so if you want to add one item, (or many), create a new tuple with the item(s), and add it to the existing tuple.

```
thistuple = ("apple", "banana", "cherry")
y = ("orange",)
thistuple += y
print(thistuple)
del thistuple
```

- The del keyword can delete the tuple completely.



Python Data Types: Tuples [11]

- Python allows us to extract values in a tuple back into variables. This is called unpacking.

```
fruits = ("apple", "banana", "cherry")
(green, yellow, red) = fruits
print(green)
print(yellow)
print(red)
```



Python Data Types: Tuples [11]

- If the number of variables is less than the number of values, you can add an $*$ to the variable name and the values will be assigned to the variable as a list.

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")
(green, yellow, *red) = fruits
print(green)
print(yellow)
print(red)
```



Python Data Types: Tuples [11]

- If the asterisk is added to another variable name than the last, Python will assign values to the variable until the number of values left matches the number of variables left.

```
fruits = ("apple", "mango", "papaya", "pineapple", "cherry")
(green, *tropic, red) = fruits
print(green)
print(red)
print(tropic)
```

- To join two or more tuples you can use the + operator. Example:

```
tuple1 = ("a", "b" , "c")
tuple2 = (1, 2, 3)
tuple3 = tuple1 + tuple2
print(tuple3)
```



Python Data Types: Tuples [11]

- If you want to multiply the content of a tuple a given number of times[8], you can use the $*$ operator.

```
fruits = ("apple", "banana", "cherry")
mytuple = fruits * 2
print(mytuple)
```

- Python has two built-in methods that you can use on tuples.

Method	Description
count()	Returns the number of times a specified value occurs in a tuple
index()	Searches the tuple for a specified value and returns the position of where it was found

Table 16: Tuple Methods



[8] Item in the tuples will be copied total n times, if you multiply by n

Python Data Types: Tuples [11]

- The method index(x) gives the index number of the first occurrence of x in the tuple.

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.count(5)
x = thistuple.index(8)
print(x)
```



Python Data Types: Sets [11]

- A set is a collection which is unordered, unchangeable[9], and unindexed.
- Sets do not allow duplicate values.
- Sets are written with curly brackets.
- The values True and 1 are considered the same value in sets, and are treated as duplicates.
- Set items can be of any data type.
- Once a set is created, you *cannot change* its items, but you *can add new items*.

```
thisset = {"apple", "banana", "cherry", True, 1, 2}
print(thisset)
print(len(thisset))
```



[9] Set items are *unchangeable*, but you can remove items and add new items

Python Data Types: Sets [11]

- It is also possible to use the set() constructor to make a set. Example

```
thisset = set(("apple", "banana", "cherry"))
print(thisset)
for x in thisset:
      print(x)
print("banana" in thisset)
```

- You cannot access items in a set by referring to an index or a key.
- But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the in keyword.



Python Data Types: Sets [11]

- To add one item to a set use the add() method.

```
thisset = {"apple", "banana", "cherry", True, 1, 2}
thisset.add("orange")
print(thisset)
tropical = {"pineapple", "mango", "papaya"}
thisset.update(tropical)
print(thisset)
```

- To add items from another set into the current set, use the update() method.
- The object in the update() method does not have to be a set, it can be any iterable object.



Python Data Types: Sets [11]

- To remove an item in a set, use the remove(), or the discard() method.
- If the item to remove does not exist, the remove() method will raise an error but the discard() method will not.
- You can also use the pop() method to remove an item, but this method will remove a random item which is also returned by this method.

```
thisset = {"apple", "banana", "cherry", True, 1, 2}
thisset.remove("banana")
print(thisset)
thisset.discard(1)
x = thisset.pop()
print(thisset)
print(x)
```



Python Data Types: Sets [11]

- The clear() method empties the set.
- The del keyword will delete the set completely.

```
thisset = {"apple", "banana", "cherry", True, 2}
thisset.clear()
print(thisset)
del thisset
```

- Joining two Sets: You can use the union() method that returns a new set containing all items from both sets.
- You can also use the update() method that inserts all the items from one set into another.



Python Data Types: Sets[11]

- Joining two Sets: You can use the union() method that returns a new set containing all items from both sets.
- You can also use the update() method that inserts all the items from one set into another.
- Both the above-discussed methods will exclude any duplicate item.

```
set1 = {"a", "b" , "c"}
set2 = {1, 2, 3}
set3 = set1.union(set2)
print(set3)
set1.update(set2)
print(set1)
```



Python Data Types: Sets [11]

- The intersection_update() method will keep only the items that are present in both sets.
- On the other hand, the intersection() method will return a new set, that only contains the items that are present in both sets.
- The symmetric_difference() method will return a new set, that contains only the elements that are NOT present in both sets.

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
x.intersection_update(y)
print(x)
z = x.intersection(y)
print(z)
d = x.symmetric_difference(y)
print(d)
```



Python Data Types: Sets [11]

Method	Description
isdisjoint()	Returns whether two sets have a intersection or not
issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
copy()	Returns a copy of the set (y=x.copy())

Table 17: Functions for Set

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
```



Python Data Types: Dictionaries [11]

- Dictionaries are used to store data values in key:value pairs.
- Dictionary items are ordered, changeable, and do not allow duplicates i.e. they cannot have two items with the same key.
- When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change.
- Dictionary items are presented in key:value pairs, and can be referred to by using key within a square bracket.

```
thisdict = { "brand": "Ford", "model": "Mustang", "year":
1964 }
print(thisdict)
print(thisdict["brand"])
```



Python Data Types: Dictionaries[11]

- Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.
- The values in dictionary items can be of any data type
- From Python's perspective, dictionaries are defined as objects with the data type dict.
- It is also possible to use the dict() constructor to make a dictionary.

```
thisdict = dict(brand = "Ford", model = 36, year = "Norway")
print(thisdict)
print(thisdict["brand"])
```



Python Data Types: Dictionaries [11]

- You can use len() function to get the number of items a dictionary has.
- You also use dictionary method get() to get a value associated with a key.
- You can get a list of all the keys (dict_keys type) in a dictionary using keys() method.
- You can get a list of all the values (dict_values type)in the dictionary using method value().

```
thisdict = { "brand": "Ford", "model": "Mustang", "year":
1964 }
print(thisdict.len())
print(thisdict.get("brand"))
x = thisdict.keys()
y = thisdict.values()
```



Python Data Types: Dictionaries [11]

- The items() method will return each item in a dictionary (dict_item type), as a list of tuples.
- You can change the value of a specific item by referring to its key name.
- You can add a new key-value pair as follows:

```
thisdict = { "brand": "Ford", "model": "Mustang", "year": 1964 }
thisdict["color"] = "white"
x = thisdict.items()
thisdict["year"] = 2018
```



Python Data Types: Dictionaries [11]

- The update() method allows you to update a dictionary by specifying items to be updated/added in the form of arguments.
- The argument must be a dictionary or an iterable object with key:value pairs.
- The pop(key) method allows you to remove an item with the specified key name.

```
thisdict = { "brand": "Tata", "model": "Nexon", "year": 2016 }
thisdict.update({"year": 2020})
thisdict.update({"County of Origin": "India"})
thisdict.pop("model")
```



Python Data Types: Dictionaries [11]

- The popitem() method removes the last inserted item.
- The del keyword removes the item with the *specified key name*.
- It can also delete the dictionary completely.
- The clear() method empties the dictionary.

```
thisdict = { "brand": "Tata", "model": "Nexon", "year": 2016 }
thisdict.popitem()
del thisdict["model"]
thisdict.clear()
del thisdict
```



Python Data Types: Dictionaries [11]

- You can loop through all the keys in a dictionary by using a for loop.
- You can also loop through all the values in the dictionary using the for loop as follows:

```
thisdict = { "brand": "Tata", "model": "Nexon", "year":
2016 }
for x in thisdict:
      print(x)
for x in thisdict:
      print(thisdict[x])
```



Python Data Types: Dictionaries [11]

- You can also use the values() method to return values of a dictionary.
- You can use the keys() method to return the keys of a dictionary.
- Loop through both keys and values, by using the items() method.

```
thisdict = { "brand": "Tata", "model": "Nexon", "year": 2016 }
for x in thisdict.values():
      print(x)
for x in thisdict.keys():
      print(x)
for x, y in thisdict.items():
      print(x, y)
```



Python Data Types: Dictionaries [11]

- You can make a copy of a dictionary with the copy() method.
- You cannot copy a dictionary simply by typing *dict2* = *dict1* (for example), because: *dict2* will only be a reference to *dict1*, and changes made in *dict1* will automatically also be made in *dict2*.
- Another way to make a copy is to use the built-in function dict().

```
thisdict = { "brand": "Tata", "model": "Nexon", "year":
2016 }
mydict = thisdict.copy()
print(mydict)
newdict = dict(mydict)
```



Python Data Types: Dictionaries [11]

- A dictionary can contain dictionaries, this is called nested dictionaries.

```
myfamily = {
"child1" : { "name" : "Amit", "year" : 2004 },
"child2" : { "name" : "Akhil", "year" : 2007 },
"child3" : { "name" : "Ramesh", "year" : 2011 }
```



Python Data Types: Dictionaries [11]

- Alternatively, you can create three dictionaries, then create one dictionary that will contain the other three dictionaries.
- To access items from a nested dictionary, you use the name of the dictionaries, starting with the outer dictionary.

```
child1 = { "name" : "Emil", "year" : 2004 }
child2 = { "name" : "Tobias", "year" : 2007 }
child3 = { "name" : "Linus", "year" : 2011 }
myfamily = { "child1" : child1, "child2" : child2, "child3" :
child3 }
print(myfamily["child2"]["name"])
```



Python Data Types: Dates [11]

- A date in Python is not a data type of its own, but we can import a module named datetime to work with dates as date objects.
- The date contains year, month, day, hour, minute, second, and microsecond.
- You can use method now() to get the current date and time.
- For example, you can import the datetime module and display the current date as follows:

```
import datetime
x = datetime.datetime.now()
print(x)
```



Python Data Types: Dates [11]

- To create a date, you can use the constructor datetime() of the datetime class which requires three parameters (year, month and day).

```
import datetime
x = datetime.datetime(2020, 5, 17)
print(x)
```



Python Data Types: Dates [11]

- The datetime object has strftime() method for formatting date objects into readable strings which takes one parameter, format, to specify the format of the returned string.
-

```
import datetime
x = datetime.datetime(2020, 5, 17)
print(x.strftime("%B"))
```



Python Data Types: Dates [11]

Directive	Description	Example
%a	Weekday, short version	Wed
%A	Weekday, full version	Wednesday
%w	Weekday as a number 0-6, 0 is Sunday	3
%d	Day of month 01-31	31
%b	Month name, short version	Dec
%B	Month name, full version	December
%m	Month as a number 01-12	12
%y	Year, short version, without century	18
%Y	Year, full version	2018
%H	Hour 00-23	17

Figure 2: The legal format codes for dates



Python Data Types: Dates [11]

%I	Hour 00-12	05
%p	AM/PM	PM
%M	Minute 00-59	41
%S	Second 00-59	08
%f	Microsecond 000000-999999	548513
%z	UTC offset	+0100
%Z	Timezone	CST
%j	Day number of year 001-366	365
%U	Week number of year, Sunday as the first day of week, 00-53	52
%W	Week number of year, Monday as the first day of week, 00-53	52
%c	Local version of date and time	Mon Dec 31 17:41:00 2018

Figure 3: The legal format codes for dates



Python Data Types: Dates [11]

%C	Century	20
%x	Local version of date	12/31/18
%X	Local version of time	17:41:00
%%	A % character	%
%G	ISO 8601 year	2018
%u	ISO 8601 weekday (1-7)	1
%V	ISO 8601 weeknumber (01-53)	01

Figure 4: The legal format codes for dates



4. Binary Types
    1. Tuples
        - It is also sequence of data similar to a list, but it is immutable[10].
        - Would be updated later.....

[10] ie. the data is write-protected



Python Functions [11]

- A function is a cohesive block of code to execute a specific task which only runs when it is called.
- You can pass data, known as parameters, into a function. A function can return data as a result.
- To call a function, use the function name followed by parenthesis.
- In Python a function is defined using the def keyword. Example:

```
def my_function():
    print("Hello from a function")
my_function()
```



Python Functions [11]

- Arguments are specified after the function name, inside the parentheses.
- You can send arguments of any data type to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.
- You can add as many arguments as you want, just separate them with a comma. Example:

```
def my_function(fname):
    print(fname + " @CUSB")
my_function("Anil")
my_function("Aman")
```



Python Functions [11]

- If you do not know how many arguments will be passed into your function, add a $*$ before the parameter name in the function definition.
- This way the function will receive a tuple of arguments and can access the items accordingly. Example:

```
def my_function(*fname):
    print("The youngest student is", +fname[2])
my_function("Anil", "Aman", "Vijay")
```



Python Functions [11]

- You can also send arguments with the key = value syntax.
- This way the order of the arguments does not matter. Example:

```
def my_function(child3, child2, child1):
    print("The youngest child is " + child3)
my_function(child1 ="Aman", child2 = "Vijay", child3 =
"Anil", )
```



Python Functions [11]

- If the number of key-value pairs of arguments is unknown, add a double $**$ before the parameter name. Example:

```
def my_function(**kid):
    print("His last name is " + kid["lname"])
my_function(fname = "Vijay" , lname = "Raj")
```



Python Functions [11]

- To let a function return a value, you can use the return statement.
- Function definitions cannot be empty, but if you for some reason have a function definition with no content, put in the pass statement to avoid getting an error.
- Python also allows recursion.

```
def my_function():
    pass
my_function()
```



Python Functions [11]

- You can send arguments of any data type to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.
- You can define a function with default parameter value. Example:

```
def my_function(country = "India"):
    print("I am from " + country)
my_function("UAE")
my_function("USA")
my_function("Australia")
my_function()
```



Python Functions [11]

- A python module is a file containing a set of python functions you want to include in your application.
- It is a shareable library of code.
- To create a module just save the code in a file with the file extension .py.
- You can create an alias when you import a module, by using the as keyword.
- You can use function dir() to list all the function names (or variable names) in a module.



Python Basics: Global and Local Variables

- A variable created in the main body of the Python code is a global variable and belongs to the global scope [11].
- By default the scope of all the variables is local to the block in which they are defined.
- For example when we define variables inside a function definition, they are local to the function by default.
- If we define a variable outside any block it becomes global variable.
- In order to use a local variable outside the block, we need to declare it with the keyword global.
- Note: To change the value of a global variable inside a function, you have to refer to the variable by using the global keyword.



Python Functions [11]

```
1   def myfunc():
2     y=40
3     x = 300
4   def myinnerfunc():
5       print(x)
6       global z
7       z=400
8       print(y)
9       myinnerfunc()
10      myfunc()
11      print(y)
12      print(z)
13
```



Python Exception Handling

- An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions[8].
- In Python, an error can be a syntax error or an exception[6].
- A Python program terminates as soon as it encounters an error[6].
- All Python exceptions inherit from the BaseException class, or extend from an inherited class therein. The full exception hierarchy of this error is as follows:[1]
- BaseException
  - Exception
  - GeneratorExit
  - KeyboardInterrupt
  - SystemExit



Python Exception Handling

Exception has following sub-classes:

1. ArithmeticError:The base class for the variety of arithmetic errors
   - FloatingPointError
   - OverflowError
   - ZeroDivisionError
2. AssertionError: This error is raised when a call to the assert statement fails.
3. AttributeError: It is raised when an attribute reference or assignment fails
4. BufferError: It is raised when a buffer-related operation cannot be performed
5. EOFError: It is raised when one of the built-in functions input() or raw_input() hits an end-of-file condition (EOF) without reading any data.



Exception has following (in-built) sub-classes.....:

6. ImportError
   - ModuleNotFoundError
7. LookupError
   - IndexError
   - KeyError
8. MemoryError
9. NameError: Raised when trying to use an identifier with an invalid or unknown name.
   - UnboundLocalError



Python Exception Handling

- OSError: This error is raised when a system-level problem occurs, such as failing to find a local file on disk or running out of disk space entirely
  - BlockingIOError
  - ChildProcessError
  - ConnectionError
    - BrokenPipeError
    - ConnectionAbortedError
    - ConnectionRefusedError
  - ConnectionResetError
  - FileExistsError
  - FileNotFoundError
  - InterruptedError
  - IsADirectoryError
  - NotADirectoryError
  - PermissionError
  - ProcessLookupError
  - TimeoutError



Python Exception Handling

- ReferenceError
- RuntimeError
  - NotImplementedError
  - RecursionError
- StopIteration
- StopAsyncIteration
- SyntaxError
  - IndentationError
    - TabError
- SystemError: A generic error that is raised when something goes wrong with the Python interpreter



Python Exception Handling

- TypeError: This error is raised when attempting to perform an operation on an incorrect object type.
- ValueError: This error is raised when a function or method receives an argument of the correct type, but with an actual value that is invalid for some reason
  - UnicodeError
    - UnicodeDecodeError
    - UnicodeEncodeError
    - UnicodeTranslateError



Python Exception Handling

- Warning: The Warning class is used to alert the user in non-dire situations
  - BytesWarning
  - DeprecationWarning
  - FutureWarning
  - ImportWarning
  - PendingDeprecationWarning
  - ResourceWarning
  - RuntimeWarning
  - SyntaxWarning
  - UnicodeWarning
  - UserWarning



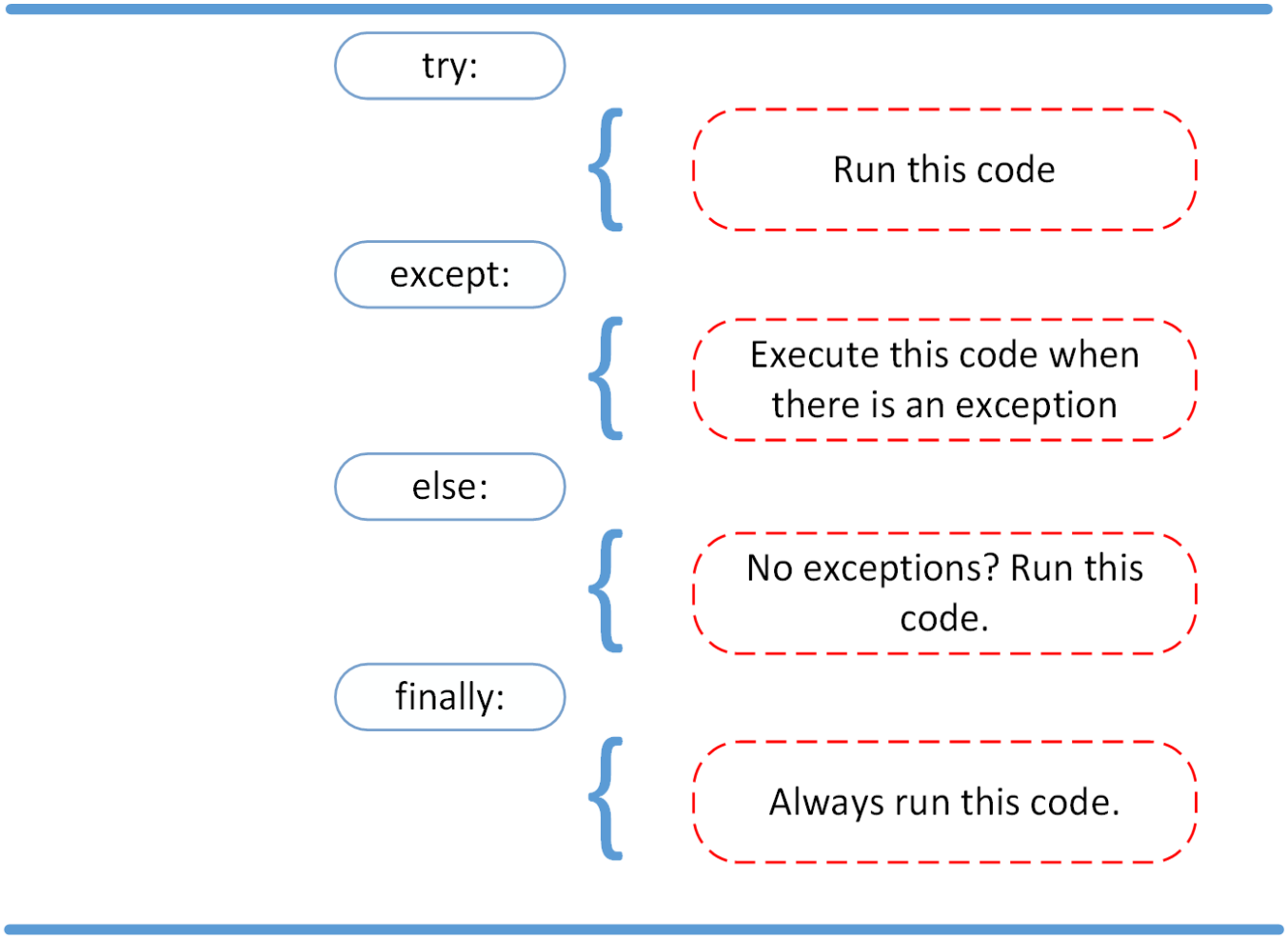


Figure 5: General Syntax for Exception Handling[7]



Python Exception Handling[7]

Raising an Exception:

- You can use keyword raise to throw an exception if a condition occurs.
- The program comes to a halt and displays the following exception to screen, offering clues about what went wrong. Exception: x should not exceed 5. The value of x was: 10

```
1    x = 10
2    if $x > 5$:
3      raise Exception('x should not exceed 5. The
     value of x was: { }'.format(x))
4
```



Python Exception Handling[7]

- The try and except block in Python is used to catch and handle exceptions.
- Python executes code following the try statement as a normal part of the program.
- The code that follows the except statement is the program's response to any exceptions in the preceding try clause.

```
1  try:
2    with open('file.log') as file:
3      readData = file.read()
4  except FileNotFoundError as fnfError:
5    print(fnfError)
6
```



Python Exception Handling[7]

- You can have more than one function call in your try clause and anticipate catching various exceptions.
- Warning: Catching Exception hides all errors, even those which are completely unexpected.
- This is why you should avoid bare except clauses in your Python programs and use specific exception classes you want to catch and handle.
- Imagine that you always had to implement some sort of action to clean up after executing your code. Python enables you to do so using the finally clause.



Python Exception Handling[7]

```
1     try:
2       linux_interaction()
3       with open('file.log') as file:
4         read_data = file.read()
5     except FileNotFoundError as fnf_error:
6       print(fnf_error)
7     except AssertionError as error:
8       print(error)
9       print('linux_interaction() function was not
      executed')
10    finally:
11      file.close()
12    print('Cleaning up, irrespective of any exceptions
      .')
13
```



Python Time Module [10]

- To measure code performance, we need to calculate the time taken by the script/program to execute.
- Time module is an in-built module that allows you to work with time.
- It allows you to represent time in various ways such as numbers, strings, floats etc.
- To use this module you need to import it as follows:
- An epoch of time can be defined as the starting point from which we can measure the passage of time.
- The point when the time begins is defined as the epoch for the Unix system as January 1, 1970, 00:00:00 at UTC.

```
1    import time
2
```



Python Time Module [2]

- We often come across the following two terms to measure the execution time
  1. Wall clock time: It is the difference between the time at which a program finished its execution and the time at which the program started. It also includes waiting time for resources.
  2. CPU time: It refers to the time the CPU was busy processing the program's instructions. It does not include the waiting time for resources.
- The difference between the Wall time and CPU time can occur from architecture and run-time dependency, e.g., programmed delays or waiting for system resources to become available.



Python Time Module [2]

- We may require to record the start time before executing the first line of the source code of the program.
- The time() function of time module is used to get the time in seconds since epoch.
- Next, we need to get the end time before executing the last line of the program to obtain the execution time of the program.
- You can use strftime() to convert the time in a more readable format like (hh-mm-ss) hours-minutes-seconds.

```
1     print('Execution time:', time.strftime("%H:%M:%S",
       time.gmtime(elapsed_time)))
2
```



Python Time Module: Using time Module

time module is simple and suitable for most use cases.

```
1      import time
2      start_time = time.time()
3      # Your function call
4      elapsed_time = time.time() - start_time
5      print(f"Elapsed time: {elapsed_time} seconds")
6
```



Using timeit Module for Single Execution

The timeit module provides more accurate timing by running the code multiple times and averaging the results.

```
1     import timeit
2     def my_function():
3     # Your function code
4     start_time = timeit.default_timer()
5     my_function()
6     elapsed_time = timeit.default_timer() - start_time
7     print(f"Elapsed time: {elapsed_time} seconds")
8
```



Using `timeit` Module for Multiple Executions

```
1     import timeit
2     def my_function():
3     # Your function code
4     # Time the function over multiple runs and get the
       average
5     elapsed_time = timeit.timeit("my_function()",
     globals=globals(), number=1000) / 1000
6     print(f"Average elapsed time: {elapsed_time}
     seconds per execution")
7
```



Using datetime Module

The datetime module provides a way to measure elapsed time with a human-readable output.

```
1      from datetime import datetime
2      start_time = datetime.now()
3      # Your function call
4      end_time = datetime.now()
5      elapsed_time = end_time - start_time
6      print(f"Elapsed time: {elapsed_time}")
7
```



Using cProfile Module

For more detailed profiling, including measuring elapsed time and identifying bottlenecks, you can use the cProfile module.

```
1     import cProfile
2     def my_function():
3     # Your function code
4     cProfile.run('my_function()')
5
```



Using perf_counter from time Module

perf_counter provides a higher resolution timer which is suitable for timing code with very fine granularity.

```
1     import time
2     start_time = time.perf_counter()
3     # Your function call
4     elapsed_time = time.perf_counter() - start_time
5     print(f"Elapsed time: {elapsed_time} seconds")
6
```



Summary

- time.time(): Easy and good for simple measurements.
- timeit: More accurate for timing small functions or code snippets, especially useful for benchmarking.
- datetime.datetime.now(): Good for getting a readable output.
- cProfile: Useful for detailed profiling and identifying performance bottlenecks.
- time.perf_counter(): High-resolution timer for more precise measurements.

Choose the method that best fits your needs based on the level of detail and precision required.



Thanks for Listening.



References I

[1] Frances Banks. *The Python Exception Class Hierarchy*. Accessed: 2023-10-06. 2017. URL: https://airbrake.io/blog/python-exception-handling/class-hierarchy.

[2] Vishal Hule. *Python Measure the Execution Time of a Program*. Accessed: 2024-05-15. 2022. URL: https://pynative.com/python-get-execution-time-of-program.

[3] Python Institute. *Python-the language of today and tomorrow*. Accessed: 2023-08-24. 2023. URL: https://pythoninstitute.org/about-python.

[4] Alice Ivey. *History of Python programming language*. Accessed: 2023-08-24. 23. URL: https://cointelegraph.com/news/history-of-python-programming-language.



References II

[5] Derrick Kearney. *First Steps With Python*. 2019. URL: https://realpython.com/python-first-steps/.

[6] Said van de Klundert. *Python Exceptions: An Introduction*. Accessed: 2023-10-06. 2023. URL: https://realpython.com/python-exceptions.

[7] Vuyisile Ndlovu. *Working With Files in Python*. 2019. URL: https://realpython.com/working-with-files-in-python/.

[8] Oracle. *What Is an Exception?* Accessed: 2023-10-06. 2023. URL: https://docs.oracle.com/javase/tutorial/essential/exceptions/definition.html.

[9] Pankaj. *Python Data Types (With Complete List)*. Accessed: 2023-08-25. 2022. URL: https://www.digitalocean.com/community/tutorials/python-data-types.



References III

[10] Srishti. *Time Module in Python*. Accessed: 2024-05-15. 2023. URL: https://www.scaler.com/topics/time-module-in-python.

[11] W3School. *Python Tutorial*. Accessed: 2023-08-26. 2023. URL: https://www.w3schools.com/python/default.asp.

